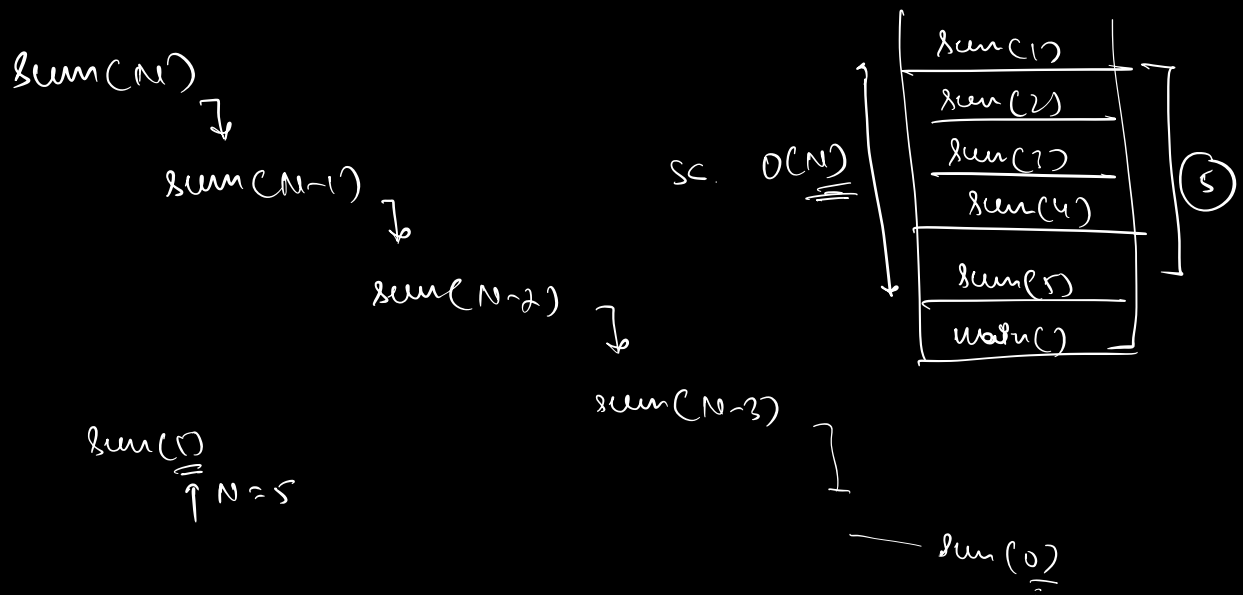


Space Complexity for recursion:-

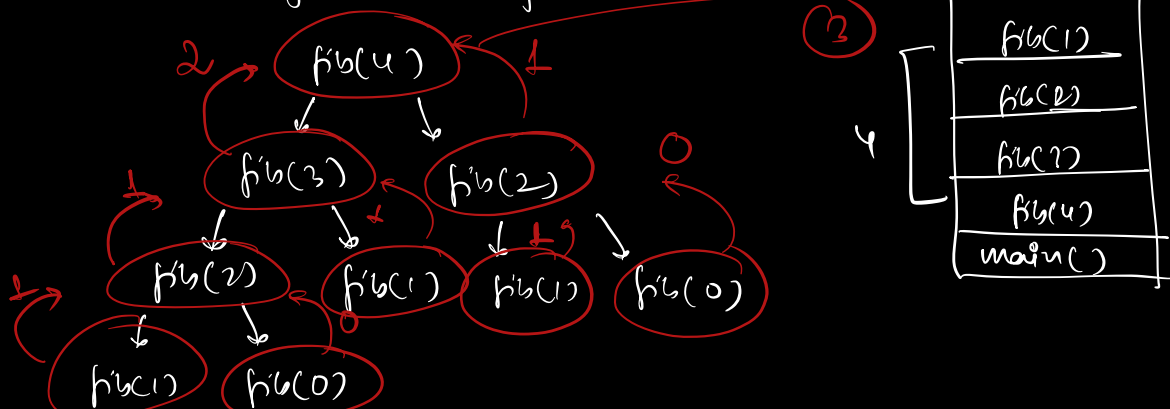


↑ max size of call stack
or, max depth of recursion

$\Rightarrow O(N)$

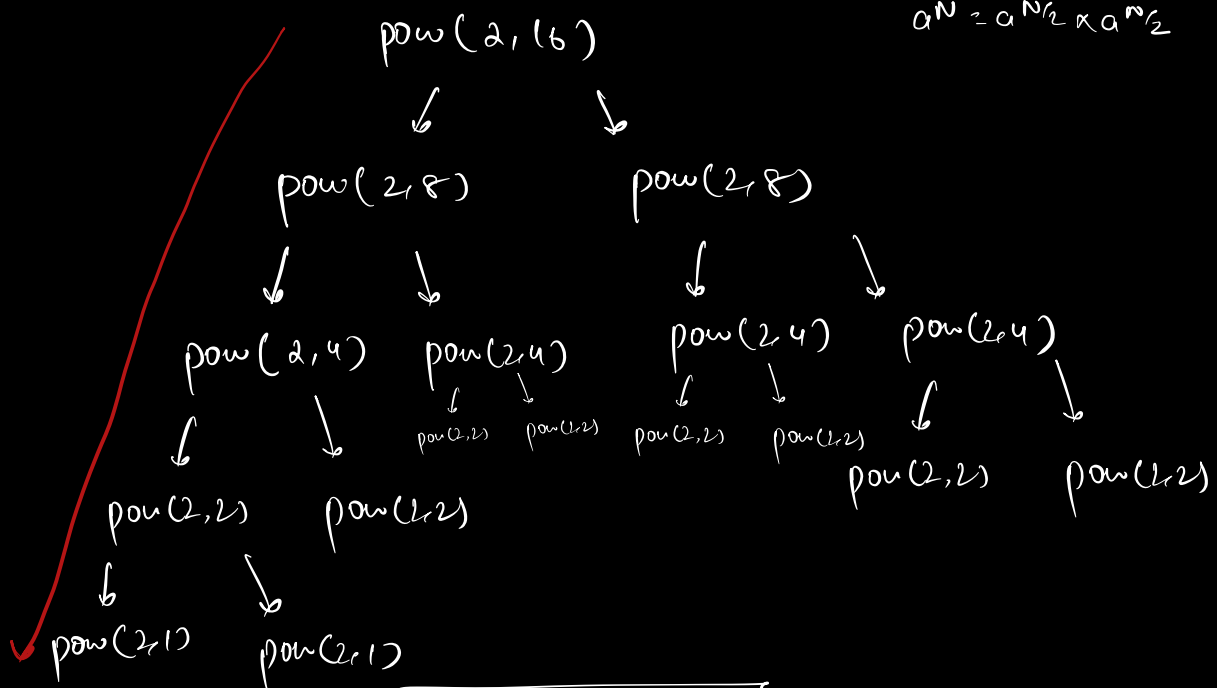
↓
Space complexity

$$\Rightarrow \text{fib}(N) = \text{fib}(N-1) + \text{fib}(N-2)$$



$$\begin{aligned}
 \text{fib}(4) &= \overset{2}{\cancel{\text{fib}(3)}} + \text{fib}(2) \\
 &\quad \swarrow \searrow \\
 &\quad \downarrow \\
 \text{fib}(3) &= \overset{1}{\cancel{\text{fib}(2)}} + \overset{1}{\cancel{\text{fib}(1)}} \\
 &\quad \downarrow \\
 \text{fib}(2) &= \overset{1}{\cancel{\text{fib}(1)}} + \overset{0}{\cancel{\text{fib}(0)}}
 \end{aligned}$$

Space comp $\Rightarrow O(N)$



SC $\Rightarrow O(\log N)$

$T_e \geq SC$

LinkedList

array

→ $O(1)$ accessing any element

$$arr[i] \Rightarrow O(1)$$

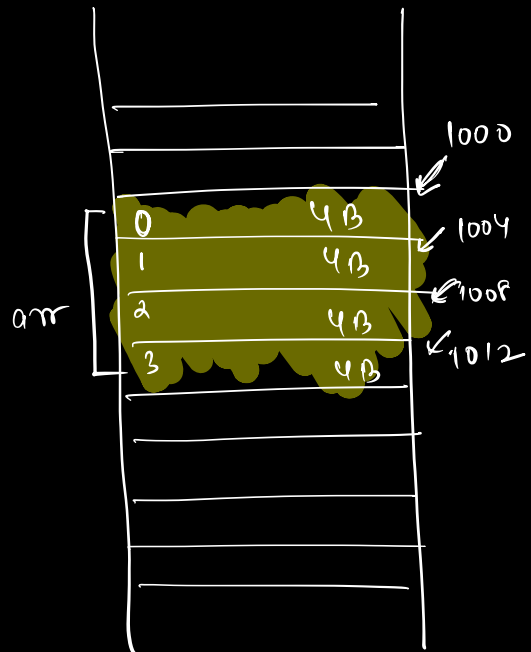
`int arr[4]`
↓
4B

stores starting
memory add.

arrays needs contiguous memory allocation

$$arr[2] \Rightarrow 1000 + (2 \times 4) \\ \Rightarrow \underline{\underline{1008}}$$

$$arr[0] \Rightarrow 1000 + (0 \times 4) \\ \Rightarrow \underline{\underline{1000}}$$



⇒ dynamic arrays (vector / arraylist) :

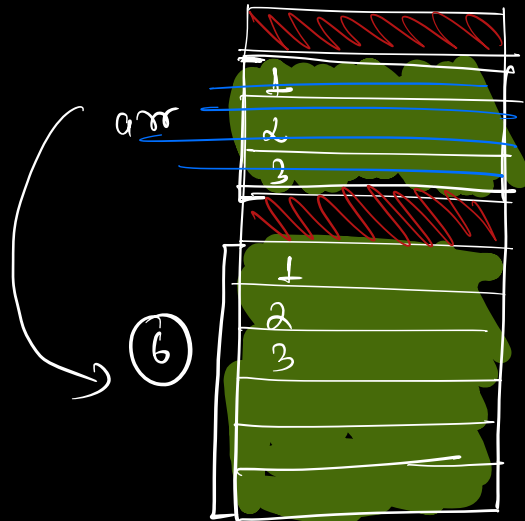
* initial size → 3 // 10

* load factor → 1 (100%) // 0.75 (75%)

* multiplicative factor → double // 1.5 times

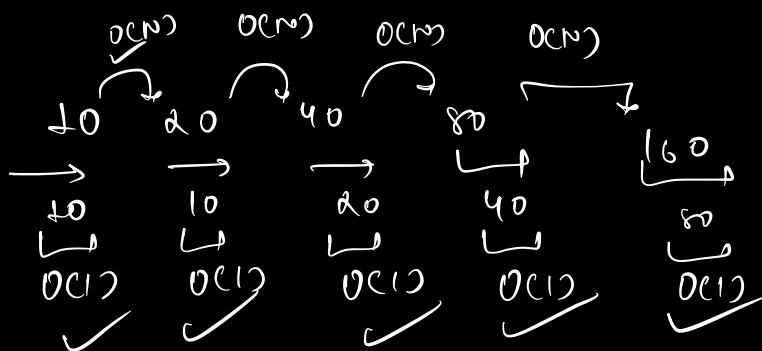
* arraylist(vector) — use array internally

arrList.put(1)
 " .put(2)
 " .put(3)
 → arrList.put(4)
 $O(N)$



10 → 20 → 40 → 80 → 160 → 320 → 640 →

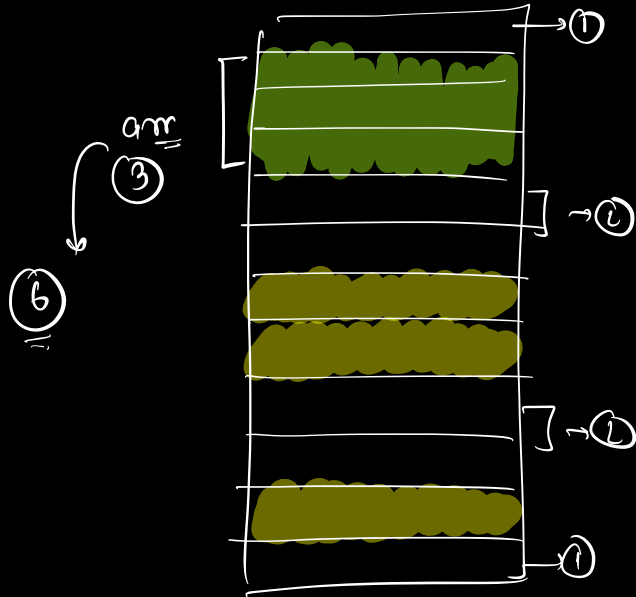
Insertion $\Rightarrow O(1) \rightarrow$ [amortised]



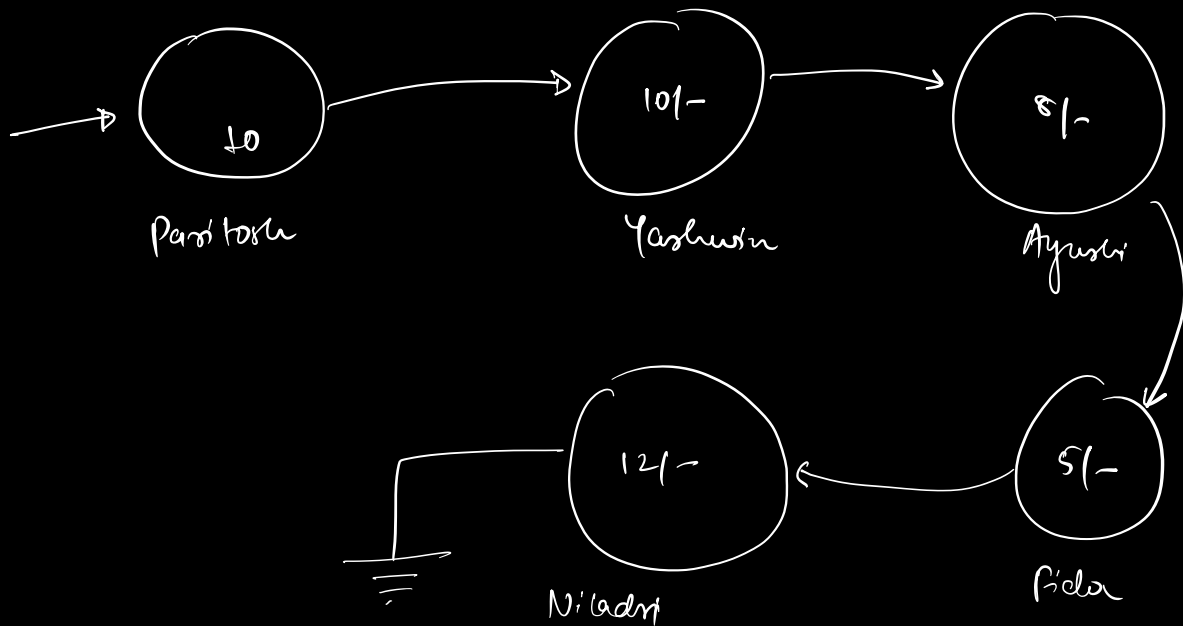
arraylist

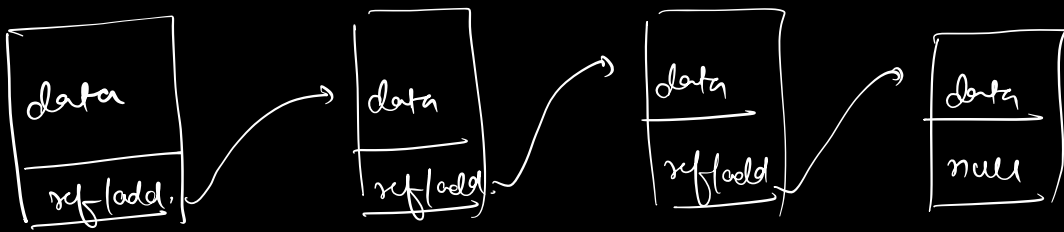
initial size = 3

6 blocks
available
but not
usable



⇒ How LL works:



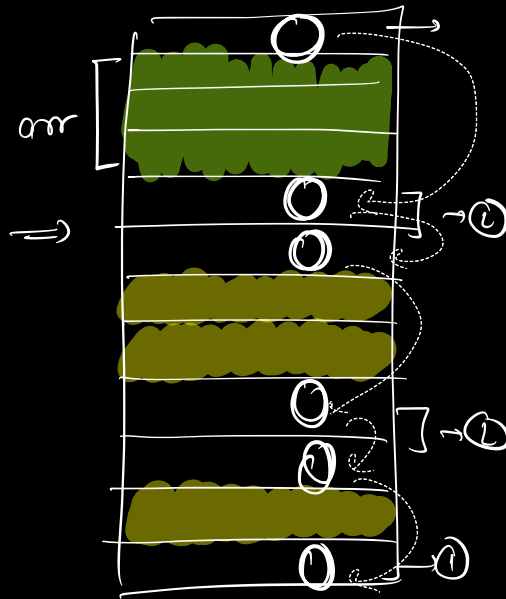


6 data pts.

random access

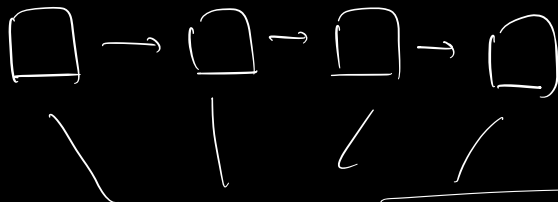
array / arraylist $\Rightarrow O(1)$

linkedlist $\Rightarrow O(N)$



Node

✓ data $\rightarrow$ int / string / arr / HashMap / Object
ref/addr.



objects
 $\Rightarrow$

```

class Node {
    int data;
    Node next;
}
  
```

Java syntax

```
Node head = new Node();
```

```
  | data →  
  | next →
```

```
public Node(a) {
```

```
  data = a;
```

```
  next = null;
```

```
Node head = new Node(10); }
```

```
  | data = 10  
  | next = null,
```

Q1. Given a LL find its length.

```
if (p == head)
```

```
  ↓  
  [10] → [20] → [30] → [40] → null
```

O/p ⇒ 4

```
int getLength(Node head) {
```

```
  int len = 0
```

```
  Node temp = head;
```

```
  while (temp != null) {
```

```
    len++
```

```
    temp = temp.next
```

```
  }
```

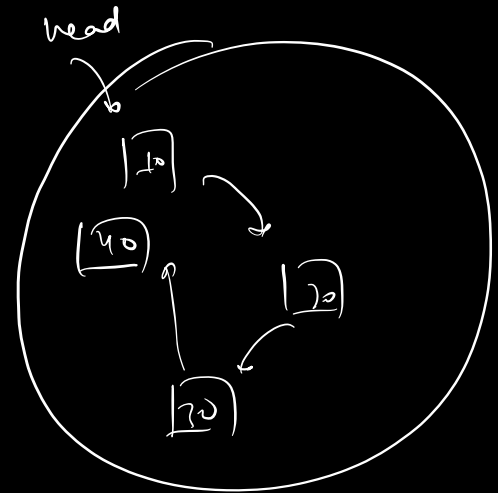
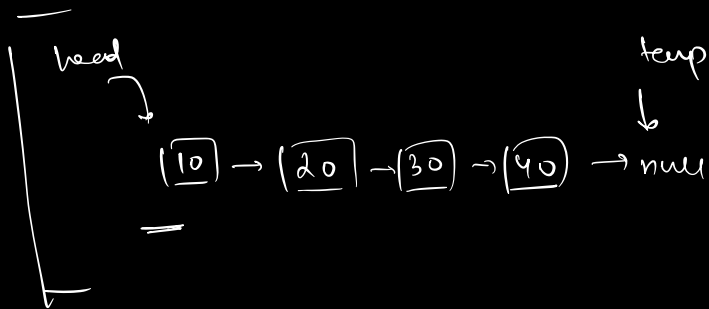
```
  return len;
```

```
}
```

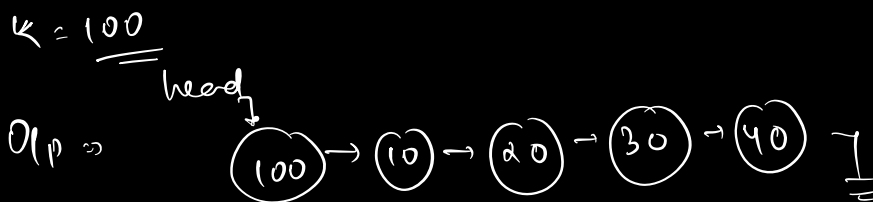
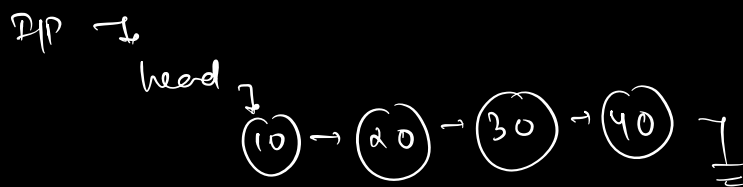
TC ⇒ $O(N)$

SC ⇒ $O(1)$

len = 4



Q2. Given a LL, add data in front

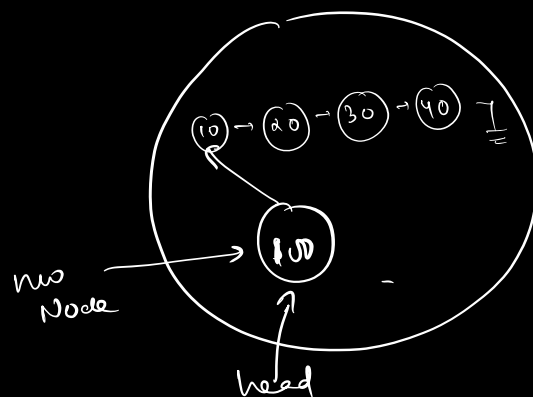


Node newNode = new Node(100);

newNode.next = head;

head = newNode;

TC = $O(1)$
SC = $O(1)$

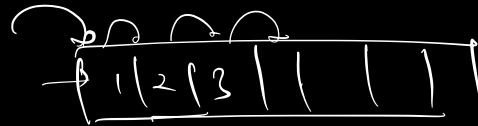


Compare

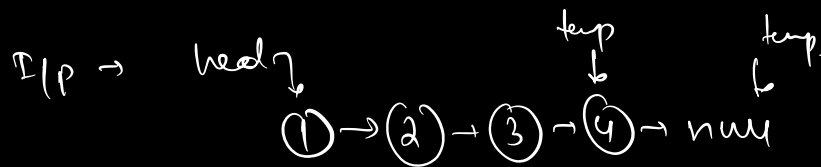
adding in front

array $\rightarrow O(N)$

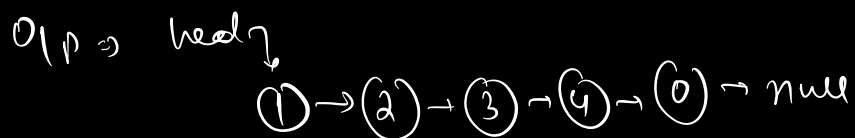
LL $\rightarrow O(1)$



Q 3. Given a LL, add data at end of LL



$k = 0$



{

Node newNode = new Node(0);

temp = head;

while (temp.next != null) {

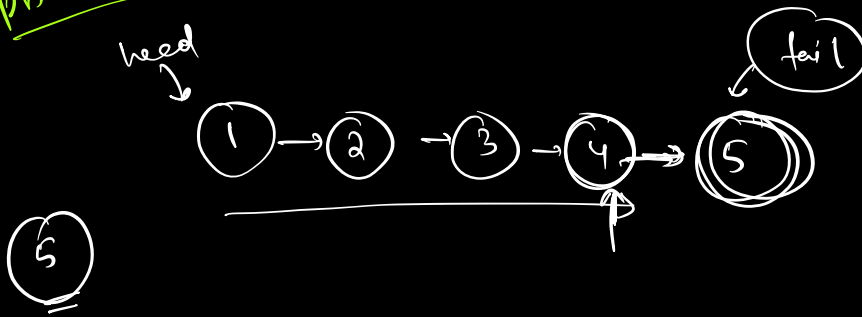
temp = temp.next;

}

temp.next = newNode;

}

Optimise

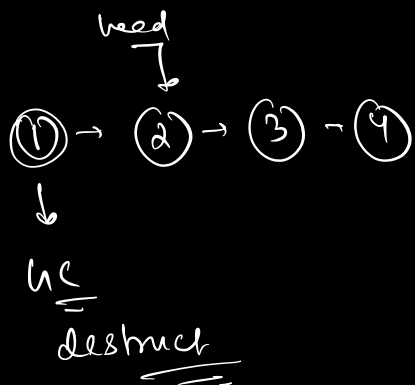


Node newNode = new Node(5)

tail.next = newNode;

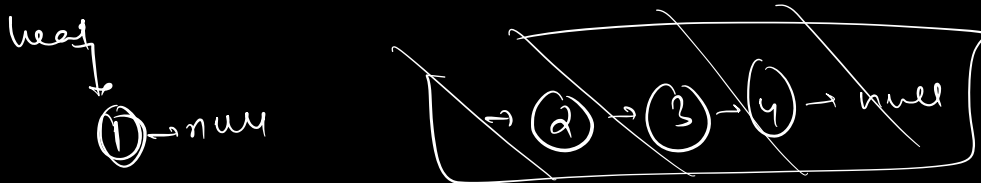
tail = newNode;

Q4, Delete a node from front of a LL.



head = head.next

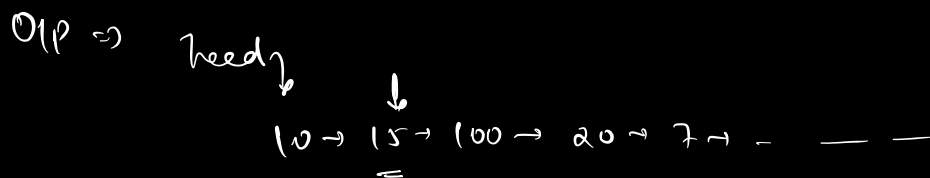
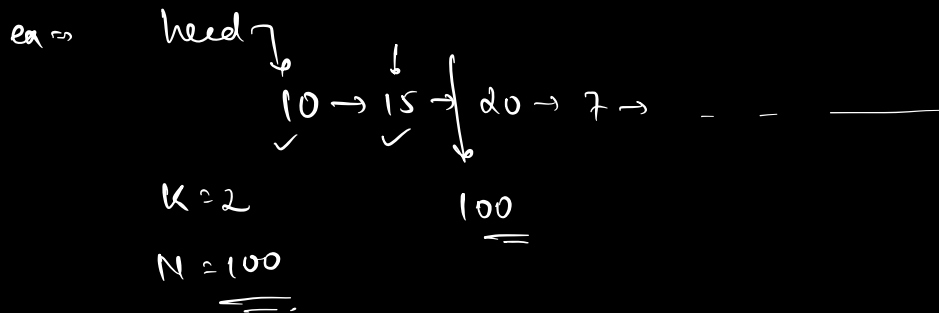
O(1)



head.next = null

* pointer manipulation *

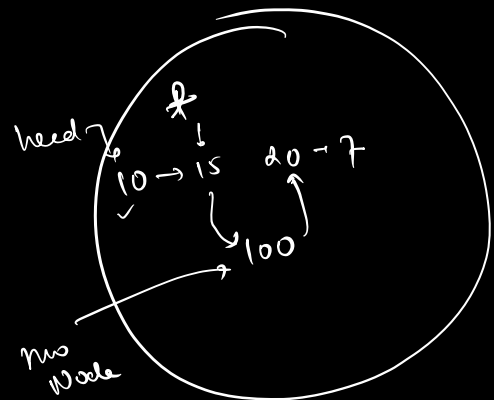
Q 4, Given a LL, Insert a node at k^{th} position.



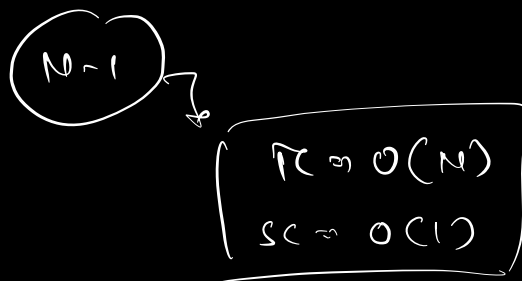
```

{
    c = 0
    temp = head
    while (c < k - 1) {
        c++;
        temp = temp->next;
    }
    newNode->next = temp->next;
    temp->next = newNode
}

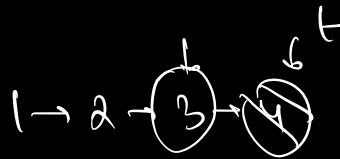
```



$10 \rightarrow 15 \rightarrow 100 \rightarrow 20 \rightarrow 7$
O/P

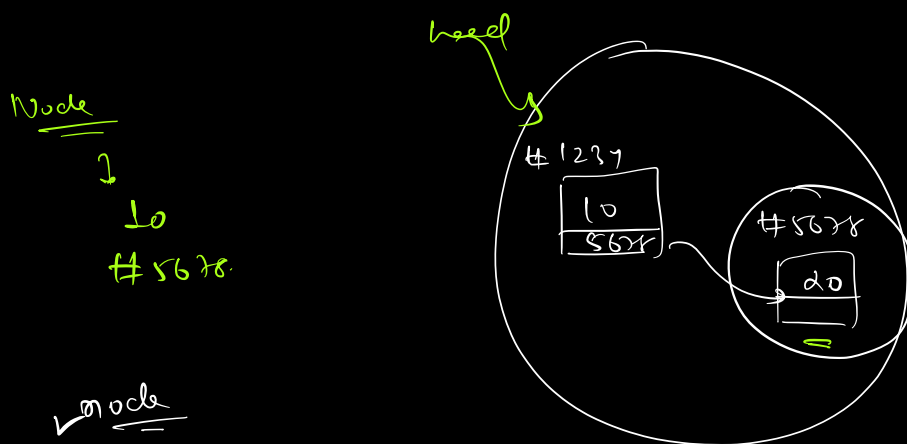


Q.11.10 Q Delete an element from end of LL.



3.next = null

1 → 2 → 3 → null



node

data = 10

next = #5678

sys: ptr (node, next)

node.next